

Research-Pi：面向计算实验科研的 Project-Centric Agent Harness

设计思想、系统实现与阶段成果汇报

Research-Pi 项目

2026 年 8 月 24 日

结论先行

Research-Pi 的核心成果不是给 Coding Agent 增加一段科研提示词，而是把长期科研状态、证据、Agent 协作、权限与 Session 交接提升为 Project 级运行时对象。Pi/DeepSeek 作为 Research Leader 负责问题理解、实验判断与用户沟通；Codex 作为可续接的 Advisor/Executor 负责工具密集和长程执行。当前系统已经形成可日常使用的科研入口，并在真实计算实验项目中持续暴露、修复上下文陈旧、跨 Session 串扰、压缩中断、权限阻塞和日志写盘等问题。工程机制已经得到自动化测试和打包验证，但“是否显著提高科研效率、缓存命中率和结论质量”仍需通过后续真实任务评估，而不能仅凭功能完成自动宣布成功。

目录

1 背景：为什么科研 Agent 不能只等同于 Coding Agent	2
2 目标与边界	2
3 总体设计原则	3
4 系统架构	4
5 关键机制	4
6 一次科研任务中的消息与判断流程	7
7 阶段成果与工程证据	7
8 真实科研使用带来的认识	8
9 当前局限与下一阶段	9
10 总结	10

1 背景：为什么科研 Agent 不能只等同于 Coding Agent

AI、通信、机器人、优化与仿真研究的大量工作已经表现为计算实验：阅读代码和论文、构造假设、实现探针、运行本地或远程实验、解释结果，再决定保留、修改或淘汰一条路线。现有 Coding Agent 往往能完成单次编码任务，却容易在长程科研中出现五类结构性问题。

1. **目标错位**：默认追求最小改动、代码整洁和生产鲁棒性，而科研早期真正需要的是快速区分假设和淘汰错误路线。
2. **上下文污染**：规划、命令、日志和中间实现都进入一个 Session，工具输出逐渐淹没研究问题。
3. **状态陈旧**：实验推进速度快于摘要和 ProjectView 更新，旧 claim 或旧 next step 可能被后续模型误当作当前事实。
4. **协作失真**：Leader 与执行 Agent 的任务、消息和完成状态容易混在一起；“子任务执行完成”又容易被误读为“研究假设成立”。
5. **权限两难**：沙箱太紧会迫使用户手工执行，完全放开又会把宿主凭据、其他项目和系统盘暴露给模型。

这些问题说明，限制长程科研表现的往往不只是模型能力，而是 Harness 如何组织状态、上下文、协作和权限。因此，Research-Pi 的研究对象可以概括为：

如何设计一个面向计算实验科研的 Agent Harness，使模型能够快速探索、可靠更新研究判断、跨 Session 保持连续性，同时不让工具噪声、旧记忆和权限摩擦主导科研过程？

2 目标与边界

Research-Pi 的目标不是自动替代研究者，也不是立即构造“全自动 AI Scientist”。当前目标有三层：

1. 让单个研究者可以把 pi 作为统一科研入口；
2. 让 Pi、Codex、实验工具和长期记忆形成可恢复、可观察的 Project Runtime；
3. 让每轮工作最终回到可解释的研究推进：支持或削弱一个假设、发现混淆因素、淘汰路线，或明确下一高信息量实验。

系统有意保留以下边界：科研结论不由退出码、提交或 Agent 的 completed 状态自动生成；高代价或不可逆决策仍由用户控制；当前实现不自动拆解所有任务、不自动选择研究路线，也不承诺固定的缓存命中率或科研提速比例。

3 总体设计原则

3.1 代码是实验工具，而不是默认需要保护的资产

研究阶段的实现首先必须正确表达实验意图。若一百行代码已经证明方向无效，整体回滚通常优于在沉没成本上继续打补丁。所谓“最小实验”指时间、算力、污染和不可逆代价小，并不要求 diff 小。只有路线获得证据支持后，才进入清理实现、补测试和提高鲁棒性的收敛阶段。

3.2 Project、Session 与 Context 分离

Research-Pi 将三个经常混淆的对象分开：

- **Project**：长期研究问题、证据、决策、Agent 和权限的共同作用域；
- **Session**：某个 Leader 的临时工作集，可以轮换或清空；
- **Context**：某一次模型调用真正需要看到的有限信息。

因此，“更换 Session”不再意味着“重新理解整个项目”，而“Project 中存在一条消息”也不意味着它应永久进入每次模型调用。

3.3 Message、State 与 Evidence 分离

消息说明发生过一次通信；State 是当前可操作的研究认知；Evidence 是能够支撑或限制研究判断的观察及其 provenance。三者的更新规则不同。一个 Codex result 可以进入 mailbox，但只有经过有效性检查的实验记录才有资格更新 claim。

3.4 沟通是科研认知接口

Leader 面向 Codex 可以使用高密度内部术语，但面向用户必须恢复足够的研究脉络。高质量汇报需要按情况选择以下语义，而不是机械套固定模板：

1. 当前结果位于哪个阶段，原本要回答什么问题；
2. Leader 与 subagent 实际做了什么；
3. 观察到了什么，关键数字相对于哪个基线或阈值；
4. 实验是否有效，主要限制是什么；
5. 证据如何改变研究判断，以及下一步为什么这样选。

当用户已经掌握上下文并继续追问时，Leader 应直接回答，不重复整段背景。这里的目标是降低重新认知成本，而不是把技术内容过度通俗化。

4 系统架构

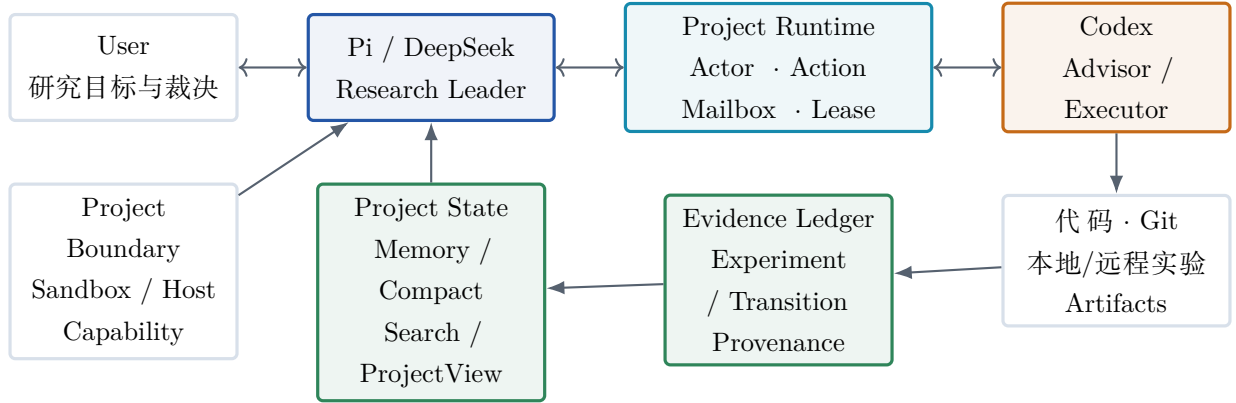


图 1: Research-Pi 的 Project-centric 分层架构。Leader 保留科研判断, Runtime 维护跨 Session 状态, Codex 隔离工具密集执行。

从实现上看, 系统建立在 Pi Core 之上, 并通过受审查的 extensions 和 libraries 形成六个层次:

1. **模型与兼容层**: 统一管理 DeepSeek/OpenCode Go profile、thinking level 和 provider compatibility;
2. **研究行为层**: 稳定 Research Contract、skill 白名单和 Research Briefing;
3. **运行时层**: Project Actor、Action、mailbox、Leader attachment 与 Session rotation;
4. **认知状态层**: experiment ledger、transition、structured compact、memory search 和 ProjectView;
5. **执行协作层**: Codex mission/thread 复用、advisor/executor、watch 与 structured result;
6. **权限层**: project sandbox、SSH target、external read、host command 与批准账本。

5 关键机制

5.1 Project Runtime: 从聊天记录转向语义事件

Runtime 采用 append-only ledger 保存少量语义事件, 而不是高频记录每个 token 或 heartbeat。设事件流为

$$L_t = (e_1, e_2, \dots, e_t), \quad S_t = \text{fold}(L_t),$$

其中 S_t 是当前 Project snapshot。只有 state commit/amend、research transition 和 evidence record 增加 Project revision; Actor activation 等运行事件不自动改变科研状态。

Project 同时只有一个 attached Research Leader Session。attachment epoch 与 activation lease 防止两个 Session 同时把自己当成 Leader; Codex completion、权限请求和消息通过 durable mailbox 交给当前 attached Session, 而不是绑定最初发起任务的聊天窗口。

5.2 ProjectView: 及时更新, 同时保护前缀缓存

早期 ProjectView 依赖 compact 后的 structured state, 实验推进后容易滞后。当前设计将它拆成:

- **structured baseline**: 最近一次可信综合, 带 session/entry/hash provenance;
- **live delta**: baseline 之后的新 transition、intervention、observation、validity 和 interpretation。

若 evidence 新于 baseline, 旧 claim 和 planned next experiment 只作为历史基线显示, 不能作为当前结论执行。窄幅更新使用 state amendment, 实质换轨使用 transition, 真正的阶段边界才进行模型 compact; 普通实验完成不机械触发压缩。

为改善 DeepSeek 的 prefix/KV cache, ProjectView 不再在每轮删除后移动到最新 user message 前, 而是在用户轮开始时通过 `before_agent_start` 向 Session 尾部持久追加 snapshot 或短 delta。同一 agent turn 内刚产生的新 evidence, 只由 context hook 在下一模型调用尾部临时补充。理想化的请求关系由

$$P_{t+1} = P_t \parallel A_t \parallel U_{t+1} \parallel \Delta V_{t+1}$$

近似表达: 旧请求 P_t 尽量保持为新请求的完整前缀, 变化集中在尾部。实际缓存仍是 provider 的 best-effort 能力, 需要根据 `cacheRead` 而非 Harness 自己的 hash 判断。DeepSeek 官方也明确不保证固定命中率。¹

5.3 Memory 与 Compact: 索引、状态和原文各司其职

Research Memory 使用本地全文索引检索历史 Session 与 experiment record, 不引入向量数据库。检索结果先返回 session/entry 引用, 再按需读取原文, 因此 assistant summary 不会天然获得比实验记录更高的证据等级。

Research Compaction 在 agent 和工具链完全 settled 后执行。默认 soft/hard 上下文阈值约为 272K/384K, 保留 24K/32K/40K 的动态 recent tail, 结构化摘要目标 8K、生成上限 16K。Compact 输出不仅是自然语言摘要, 还包括 research question、竞争假设、观察、有效性、决策、未决混淆因素、下一实验和 evidence references。完整 JSONL 历史仍然保留。

5.4 Codex 协作: 隔离执行噪声, 而不是追求形式并行

Pi 是 Leader, Codex 是具备同等项目执行权限的 operational executor 或只读 advisor。派遣的主要价值是隔离大量工具调用和中间输出, 而不是为了并发本身。

连续子任务使用稳定 mission 标签, 并在同一 workspace、mode、mission 和 research track 上复用 Codex thread。Runtime 将 transport lifecycle 与任务语义结果分开: App Server 的 completed 只说明 turn 正常结算; 只有 executor 返回 `outcome=succeeded` 且 `goal_satisfied=true`, 才表示委派目标完成。即便如此, 它仍不是科学假设成立的证据。

Codex 可在运行中发送 progress、ask 和 result。权限请求通过同一 host capability bridge 弹到 Pi TUI, 用户批准后恢复原 tool call; 完成结果携带 summary、actions、changed files、checks、

¹DeepSeek Context Caching: https://api-docs.deepseek.com/guides/kv_cache/

external effects、uncertainties 和 remaining work，Leader 再将其翻译成面向用户的研究汇报。

5.5 权限模型：项目内高自主，项目外显式越界

Research-Pi 不按 `python3 -c`、`bash -lc` 等字符串形式机械封禁命令，而按效果边界管理权限：

- 当前项目内允许正常读写、删除、Git commit、依赖安装和实验执行；
- 必要系统运行时只读开放，Git hooks 保持只读；
- 其他项目、宿主凭据、SSH agent 和 Unix socket 默认不可见；
- 项目外只读文件、精确 SSH target 和 host command 通过一次、Session 或 Project 范围批准；
- 私钥、API key、`.env` 和 keychain 不能被授权成模型可读内容。

这使“自动化执行”和“宿主安全”不再被简化成全开或全关。WSL2 场景另有 Windows 专用分支，用于限制跨入 `/mnt` 的高风险 Windows 文件操作。

5.6 低写盘观察性

Tool Activity、Runtime Dock 和 `/watch` 从已有状态投影运行进度，不建立第二套 heartbeat 日志。Codex job 只在可见进度或语义状态变化时写入；token delta storm 不持续落盘。敏感 trace 与 Codex TRACE/DEBUG SQLite 日志默认关闭，只有显式诊断时启用。

6 一次科研任务中的消息与判断流程

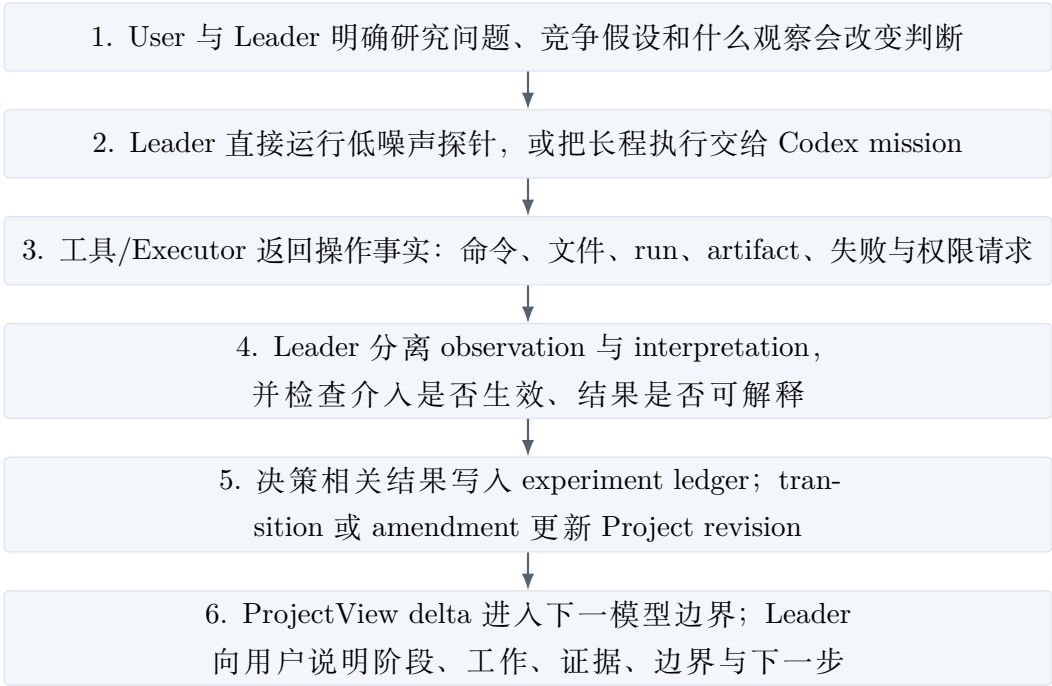


图 2: 从任务执行到科研判断的完整链路。关键点是操作事实不能跨过有效性检查直接成为结论。

7 阶段成果与工程证据

截至本报告生成时，主分支已经形成全局可安装的 pi CLI、16 个显式 Harness extensions、20 个核心 library modules 和 143 个自动化测试用例。稳定包将程序、用户配置和运行状态分离，并检查 npm manifest 不包含 credentials、sessions、traces、Runtime ledger 或 Codex job。

表 1: 主要问题、当前机制与已获得的工程证据

问题	当前机制	当前证据与边界
偏开发、过度保守	Research Contract; 探索/收敛分阶段	Prompt、skill 和真实任务行为已迭代; 尚无正式行为 A/B
长上下文压缩失真	Structured compact、provenance、recent tail、settled 后压缩	schema/unit tests 与长期 Session 实测; 摘要仍可能犯错, 原文保留
ProjectView 陈旧	revision、baseline/delta、追加式 snapshot	stale/transition/Git/大视图截断测试; 真实缓存收益待测
Codex 串话或误路由	任务身份五元组与 Leader lease	跨 Session、branch、workspace 和 thread reuse 测试
Codex 运行不可见	Runtime Dock、mailbox、watch 与结构化终态	命令/文件/ask/result 投影与低写盘测试
权限过紧导致人工接管	项目沙箱、capability broker 与 Codex 审批桥	Git/Python/SSH/host-command/preflight 测试; 高权限操作仍需人审
用户看不懂阶段进展	Leader 沟通约定与 research-briefing	结构与触发规则已实现; 表达质量需继续从真实对话迭代

工程验证包括语法检查、Runtime/ProjectView/Codex/permission 单元测试、包清单审计和隔离配置下的 CLI smoke。这里必须区分两类结论：这些检查能够说明实现路径按预期工作，却不能单独证明 Research-Pi 已经提升了科学发现质量。

8 真实科研使用带来的认识

Research-Pi 并非闭门一次性设计，而是在 EmbeddingWorld 等长程计算实验中不断被真实摩擦修正。典型例子包括：

- 两个 Pi Session 的 Codex 消息曾发生交叉，推动 job ownership 从会话局部升级为 Project Actor 与 attachment lease；
- compaction 曾在长任务结束时中断主流程，推动 “agent settled 后再 compact” 和恢复机制；
- ProjectView 落后于实验推进，推动 semantic revision、stale baseline 和 evidence delta；
- 远程脚本、系统 Git/Python 与 SSH 曾被沙箱过度限制，推动 capability broker 与 preflight；
- trace 和 Codex 内部日志可能造成高频写盘，推动稀疏语义事件、按需 watch 和默认关闭 DEBUG 日志；
- Leader 汇报充满内部黑话，推动将 “用户共享研究状态” 与 “Agent 内部高密度通信” 分开设计。

这些案例共同说明：Harness 的价值不只是提供更多工具，而是决定什么信息进入模型、何时

更新、由谁解释、以什么权限执行，以及如何让研究者持续参与高质量判断。

9 当前局限与下一阶段

9.1 尚未完成的验证

当前最重要的缺口不是继续堆叠功能，而是建立真实科研任务上的评估：

1. 对比原始 Pi、Research-Pi 以及不同 ProjectView 策略下的任务恢复时间、重复工具调用和错误路线延续率；
2. 使用 provider 返回的 hit/miss tokens 观察 append-only delta 对缓存命中率和费用的实际影响；
3. 记录用户为理解当前阶段所需的追问次数，评估 Research Briefing 是否降低认知恢复成本；
4. 衡量 experiment record 和 validity separation 是否减少“无效负结果被当作研究反证”；
5. 在 WSL2/Windows 物理机继续验证 /mnt 边界、系统盘保护和跨平台安装。

9.2 可能形成的进一步研究问题

从系统实践中可以抽象出三个具有研究价值的方向：

- **Context-native Runtime**：把状态物化、消息路由和 KV-cache locality 共同设计，而不是把 context 当作无结构字符串；
- **多 Actor 语义一致性**：在并行或异步 Agent 中保证任务、证据和研究路线不串扰，同时避免全局锁拖慢探索；
- **Human-AI Shared Research State**：把沟通质量视为科研系统性能的一部分，研究怎样让用户以较低认知成本保持有效监督和提出高价值问题。

10 总结

Research-Pi 当前已经从“给 Pi 加科研 prompt”的最初想法，发展为一个包含研究行为、Project Runtime、结构化记忆、Agent 协作、权限和用户沟通的完整 Harness。其最重要的设计思想可以归纳为：

1. 用 Project 保存长期科研认知，用 Session 承载可替换的局部工作；
2. 用语义事件和 evidence provenance 更新状态，而不是让聊天摘要垄断记忆；
3. 让 Pi 负责问题与解释，让 Codex 负责隔离执行，但不把执行完成当作科学结论；
4. 项目内给予高自主，项目外通过明确 capability 越界；
5. 把 Leader 对用户的汇报视为共享研究状态的维护，而不是附属文案；
6. 在真实科研任务中持续迭代，并对尚未验证的效果保持明确边界。

因此，现阶段最合理的评价不是“Research-Pi 已经成为自动科学家”，而是：它已经建立了一个更适合长期计算实验科研的可用运行时基础，并把下一阶段真正需要验证的问题暴露得更清楚。

项目入口与参考资料

- Research-Pi repository: <https://github.com/RosMarinas/Research-Pi>
- Pi upstream: <https://github.com/earendil-works/pi>
- DeepSeek Context Caching: https://api-docs.deepseek.com/guides/kv_cache/
- GitHub Spec Kit clarify template (沟通设计参考之一) : <https://github.com/github/spec-kit/blob/main/templates/commands/clarify.md>
- 项目内使用说明: docs/pi-basic-guide.md
- 统一配置说明: docs/configuration.md
- Runtime 测试与恢复: docs/research-runtime-test-guide.md